PitchPulse360
Live demonstration script

Three presenters | No PPT | Target: approximately 8 minutes including screen changes

How to use this script

Read the paragraphs on pages 2-4 aloud. Green SHOW / DO lines are silent instructions. Pages 5-7 are preparation and backup material, outside the timed script. Assign the presenter roles among the three members who will speak.

Presenter	Time	Responsibility
1	0:00-2:30	Problem, source, cleaning, quality and relational model
2	2:30-5:00	Lakehouse, analysis, website and benchmark
3	5:00-8:00	Prefect, API, RabbitMQ and conclusion

Open these before presenting

Football website: http://127.0.0.1:8502/

Prefect: http://127.0.0.1:4201/

API documentation: http://127.0.0.1:8002/docs

RabbitMQ queues: http://127.0.0.1:15673/#/queues

VS Code: flow.py, cleaning.py, postgres_schema.sql and dashboard app.py.

Saved evidence: results/quality/validation_report.html and results/benchmark/comparison.png.

The project in one sentence

We convert historical football CSV files into validated storage and reproducible analytical results, then present them through an interactive website.

Main route

CSV > cleaning > quality gate > storage and analytical layers > DuckDB queries > Streamlit website

Start Docker and the project in advance. Rehearse switching screens. Do not rerun the full batch during the timed demonstration. The optional event test on page 7 adds about 1-2 minutes.

Presenter 1

0:00-2:30 | Explain the problem and establish that the data is trustworthy

SHOW / DO: Open the football website. Point to the league, season and KPI cards.

Our project is PitchPulse360, a football data engineering and analytics platform. It processes 10,781 historical matches from five European leagues across six seasons, from 2018/19 to 2023/24.

The website is the final output. Behind it, we implement ingestion, cleaning, validation, structured and document storage, analytical processing, workflow automation and event capture.

Football data is spread across league and season files. Analyzing those files independently can produce inconsistent names, data types and calculations. Our pipeline centralizes these rules.

SHOW / DO: Change the competition once. Then show a raw CSV and the cleaning function.

We process thirty league-season files, reading cached inputs or downloading a missing configured historical file. Every record retains its source filename and row number for traceability. This is a historical archive, not a live-score feed.

Pandas standardizes names, parses dates and numbers, detects duplicate fixtures and checks score consistency. We derive total goals, points and goal difference, and generate a stable fixture identifier.

Invalid required scores can reject a fixture. Inconsistent optional measurements are cleared to unknown and audited instead of being replaced with invented zeros. Valid high-scoring outliers are retained.

SHOW / DO: Show the clean quality report, then postgres_schema.sql.

A forty-five-rule expectation suite acts as a gate before loading. All forty-five checks pass on the cleaned dataset. A separate damaged sample demonstrates nine detected failures.

PostgreSQL stores countries, leagues, seasons, teams and matches using keys and constraints. One match row represents one fixture. Transactional loading prevents a failed relational load from committing a partly refreshed snapshot.

HANDOFF: The next part explains how the validated records become analytical results and the website.

Presenter 2

2:30-5:00 | Connect storage, calculations and the user-facing result

SHOW / DO: Show the Bronze, Silver and Gold folders or the lakehouse code.

We organize analytical data into Bronze, Silver and Gold. Bronze preserves original records, Silver contains cleaned matches, and Gold contains aggregated team-season standings.

These are Delta tables backed by Parquet. Parquet provides columnar storage. Delta adds transaction-log-based table versions and historical reads.

A separate experiment creates, appends, updates and deletes records, producing versions zero through three. It reads the original version and rejects incompatible writes without creating another version. The experiment does not edit historical match results.

SHOW / DO: Return to the website Overview tab and point to standings and points.

The website loads the prepared analytical layers and registers the data with DuckDB for SQL queries. Streamlit provides the interface and Plotly draws the interactive charts. Filter changes do not rerun the complete pipeline.

Club points equal three times wins plus draws. Goal difference is goals scored minus goals conceded. Standings use our included results; official deductions and some league-specific rules are not fully modeled.

SHOW / DO: Show Head-to-head, then Season records. Briefly identify Player studio.

Head-to-head compares historical meetings between clubs. Season records highlights high-scoring fixtures and outcome patterns. Player movement points are generated demonstrations, not real tracking-provider data.

Our SQL work also includes reusable views, ranking and running totals, plus ROLLUP, CUBE and GROUPING SETS. Revenue examples use six thousand explicitly generated ticket orders, not real club finances.

SHOW / DO: Show the saved benchmark chart.

We compare CSV, Parquet and DuckDB using the same 250,000 repeated fixture rows. Each configuration receives a warm-up and five measurements with one-thread and four-thread limits. We verify equal answers first. This is local parallel processing, not a distributed cluster.

HANDOFF: The final part shows how we automate the workflow and capture arriving events.

Presenter 3

5:00-8:00 | Demonstrate orchestration, event reliability and the complete platform

SHOW / DO: Open Prefect. Show completed runs and task states; use flow.py for the sequence.

Prefect coordinates eleven tasks, from extraction and cleaning through validation, storage, analytical experiments and streaming verification. The current implementation executes them sequentially.

It records task states and logs, applies configured retries and schedules the workflow hourly while the services are running. A lock prevents overlapping runs from replacing outputs together. We chose Prefect for its Python-native structure and manageable local deployment.

The hourly schedule reruns the configured historical workflow. It does not automatically add new seasons or fetch today's results.

SHOW / DO: Open API documentation. Identify POST /lab/events and GET /lab/events.

FastAPI provides typed request validation and interactive API documentation. A client submits match JSON, the API validates it and publishes it to RabbitMQ. HTTP 202 means that the broker accepted the event; database processing happens asynchronously.

SHOW / DO: Open RabbitMQ Queues and Streams. Point to the normal and rejected queues.

RabbitMQ buffers messages between the API and the consumer. The consumer validates an event, commits it to PostgreSQL staging and then acknowledges it.

A unique event ID prevents repeated delivery from creating duplicate staging rows. Invalid events are recorded separately. This is at-least-once delivery with an idempotent database insert. A queue with zero waiting messages can mean the consumer has already processed them.

Events enter staging; they do not automatically rewrite historical standings. That requires an additional reconciliation step.

SHOW / DO: Return to the football website for the closing statement.

Docker Compose runs the databases and application services together. Persistent volumes preserve their data between ordinary stops and starts. MongoDB additionally preserves source documents and demonstrates nested CRUD operations and indexed queries.

Our project connects all fifteen experiments. The main outcome is traceability: we can identify where a record came from, how it was cleaned, whether it passed validation, where it was stored and how the displayed result was calculated.

Future work includes a current-results source, complete official league rules, incremental processing, access control and reconciliation of staged events into published results.

Why this technology?

Backup answers | Use when the professor asks about design choices

Choice	Why we used it / alternative considered
Python + Pandas	Convenient cleaning, profiling and integrations. SQL alone would not handle the complete application workflow as directly.
PostgreSQL	Relational integrity, transactions, analytical SQL and partitioning experiments. SQLite would simplify setup but would not demonstrate the selected server-based design.
MongoDB	Flexible source documents and nested CRUD. PostgreSQL alone would not demonstrate the separate NoSQL experiment.
DuckDB	Local analytical SQL without cluster setup. Spark/Hadoop overhead is unnecessary for this workload.
Parquet + Delta	Parquet provides typed columnar files; Delta adds a transaction log, version history and schema enforcement. Plain CSV/Parquet alone does not provide that Delta history.
RabbitMQ	A compact queued ingestion route with delivery acknowledgements. Kafka is more suited to retained event-log and replay-oriented requirements.
FastAPI	Typed payload validation and generated API documentation. Flask could work, with more manual setup for equivalent behavior.
Prefect	Python-native tasks, retries and scheduling. Airflow is valid, but adds operational structure not needed here.
Streamlit + Plotly	Direct integration with Python analytics and quick interactive visualization. React would add a separate frontend development layer.
Docker Compose	Repeatable local multi-service startup. Kubernetes would add complexity beyond this single-machine demonstration.

If asked whether the entire stack is necessary

A smaller stack could process 10,781 matches. We use these components to demonstrate the fifteen syllabus experiments, with a defined responsibility for each. We do not claim that this data volume requires enterprise-scale infrastructure.

Viva quick answers

Short, technically accurate responses | Outside the timed script

Is this just a dashboard?

No. The dashboard is the serving layer above ingestion, cleaning, validation, modeling, storage, orchestration and event processing.

What is the grain and why normalize?

The matches table has one row per fixture. Separate entities and foreign keys keep descriptive relationships consistent.

Cleaning versus validation?

Cleaning standardizes or repairs values. Validation checks the resulting data against explicit rules before publication.

Why keep outliers and unknown values?

An unusual valid result is retained. An unknown measurement remains unknown instead of being treated as an observed zero.

Do views guarantee faster queries?

No. Ordinary views save a reusable definition; they do not store a precomputed result.

What happens after a duplicate or consumer failure?

A committed event ID cannot create another staging row. Redelivery is safe for that insert because acknowledgement follows the commit.

Is the whole pipeline atomic?

No. PostgreSQL loading is transactional, but PostgreSQL, MongoDB, Delta and report outputs do not share one distributed transaction.

What does four threads mean?

A DuckDB execution limit on one machine. It does not mean four computers or guarantee a speed improvement.

Which data is generated?

The 6,000 ticket orders, player movement demonstrations and repeated benchmark workload. Event examples are replayed or manually submitted through real services.

What has been verified?

Saved evidence records 30 passing automated tests, 45 clean expectations, nine expected damaged-sample failures, reconciled match counts and eleven completed Prefect tasks.

Optional live event test

Adds approximately 1-2 minutes | Rehearse before presenting

Run the following in PowerShell while the API, RabbitMQ, consumer and PostgreSQL are running. This submits one clearly labeled demonstration event.

```
$demoEventId = [guid]::NewGuid().ToString()
$demoPayload = @{
    event_id   = $demoEventId
    demo_run   = "classroom-demonstration"
    match_id   = "classroom-demo-fixture"
    match_date = "2024-05-19"
    home_team  = "Demo Home"
    away_team  = "Demo Away"
    home_goals = 2
    away_goals = 1
    result     = "H"
} | ConvertTo-Json

Invoke-RestMethod `
    -Method Post `
    -Uri "http://127.0.0.1:8002/lab/events" `
    -ContentType "application/json" `
    -Body $demoPayload

Invoke-RestMethod `
    -Uri "http://127.0.0.1:8002/lab/events" |
    ConvertTo-Json -Depth 6
```

What to say

The API accepted this event after broker confirmation. The consumer processes it asynchronously and saves it to PostgreSQL staging. The GET request shows recent staged events; the new event may take a moment to appear.

Duplicate and validation demonstrations

Submit the same payload again without generating another event ID. It can be accepted again by the API, but should not create a second staging row. Then change result to A while keeping the 2-1 score, rebuild the JSON and submit it. The API should reject the inconsistent request with HTTP 422.

Do not expect the normal queue to build up: the consumer may process messages immediately. These demonstration events do not alter the historical league tables.

Before the presentation

Start Docker and run Open_Football.cmd from the project folder. Confirm the website and service screens load. Keep saved evidence ready in case a service is unavailable. The optional event test is not required for the core eight-minute script.